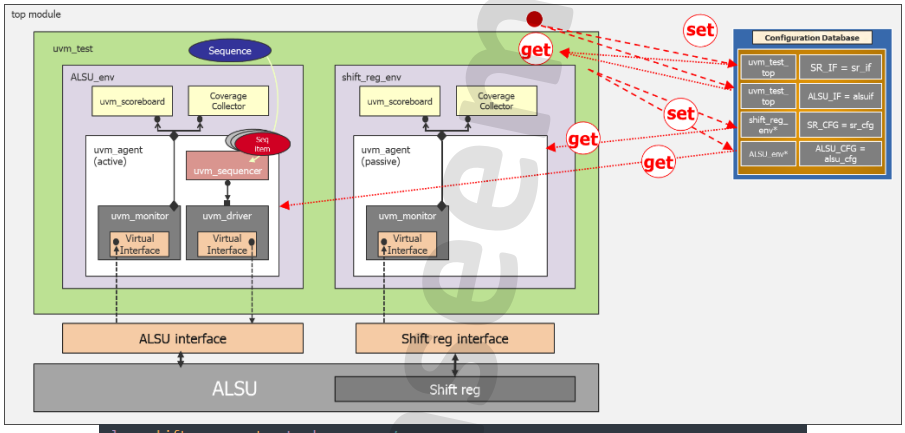
Assignment 6 [Extra] – Soln




```
class shift_reg_agent extends uvm_agent;
  `uvm_component_utils(shift_reg_agent)
   shift_reg_sequencer sqr;
   shift_reg_driver drv;
   shift_reg_monitor mon;
   shift_reg_config shift_reg_cfg;
   uvm_analysis_port #(shift_reg_seq_item) agt_ap;

  function new (string name = "shift_reg_agent", uvm_component parent = null);
   super.new(name,parent);
  endfunction

  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    if (!uvm_config_db #(shift_reg_config)::get(this , "", "CFG_SHIFT", shift_reg_cfg))begin
      `uvm_fatal("build_phase" , "Unable to get configuration object")
    end
    if (shift_reg_cfg.is_active == UVM_ACTIVE) begin
      sqr = shift_reg_sequencer::type_id::create("sqr",this);
      drv = shift_reg_driver::type_id::create("drv",this);
    end
    mon = shift_reg_monitor::type_id::create("mon",this);
    agt_ap = new("agt_ap", this);
  endfunction

  function void connect_phase(uvm_phase phase);
    if (shift_reg_cfg.is_active == UVM_ACTIVE) begin
      drv.shift_reg_vif = shift_reg_cfg.shift_reg_vif;
      drv.seq_item_port.connect(sqr.seq_item_export);
    end
    mon.shift_reg_vif = shift_reg_cfg.shift_reg_vif;
    mon.mon_ap.connect(agt_ap);
  endfunction
endclass
```

```
class shift_reg_config extends uvm_object;
  `uvm_object_utils(shift_reg_config)

  virtual shift_reg_if shift_reg_vif;
  uvm_active_passive_enum is_active;

  function new(string name = "shift_reg_config");
    super.new(name);
  endfunction
endclass
```

```
class shift_reg_driver extends uvm_driver #(shift_reg_seq_item);
  `uvm_component_utils(shift_reg_driver)

  virtual shift_reg_if shift_reg_vif;
  shift_reg_seq_item stim_seq_item;

  function new(string name = "shift_reg_driver", uvm_component parent = null);
     super.new(name, parent);
  endfunction

  task run_phase(uvm_phase phase);
    super.run_phase(phase);
    forever begin
      stim_seq_item = shift_reg_seq_item::type_id::create("stim_seq_item");
      seq_item_port.get_next_item(stim_seq_item);
      shift_reg_vif.direction = stim_seq_item.direction;
      shift_reg_vif.mode = stim_seq_item.mode;
      shift_reg_vif.datain = stim_seq_item.datain;
      shift_reg_vif.serial_in = stim_seq_item.serial_in;
      #2;
      seq_item_port.item_done();
      `uvm_info("run_phase", stim_seq_item.convert2string_stimulus(), UVM_HIGH)
    end
  endtask

endclass
```

```
class shift_reg_monitor extends uvm_monitor;
  `uvm_component_utils(shift_reg_monitor)
  virtual shift_reg_if shift_reg_vif;
  shift_reg_seq_item rsp_seq_item;
  uvm_analysis_port #(shift_reg_seq_item) mon_ap;

 function new(string name = "shift_reg_monitor", uvm_component parent = null) ;
   super.new(name, parent);
 endfunction

  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    mon_ap = new("mon_ap", this);
  endfunction: build_phase

  task run_phase(uvm_phase phase);
    super.run_phase(phase);
    forever begin
      rsp_seq_item = shift_reg_seq_item::type_id::create("rsp_seq_item");
      #2;
      rsp_seq_item.direction = direction_e'(shift_reg_vif.direction);
      rsp_seq_item.mode = mode_e'(shift_reg_vif.mode);
      rsp_seq_item.datain = shift_reg_vif.datain;
      rsp_seq_item.serial_in = shift_reg_vif.serial_in;
      rsp_seq_item.dataout = shift_reg_vif.dataout;
      mon_ap.write(rsp_seq_item);
      `uvm_info("run_phase", rsp_seq_item.convert2string(), UVM_HIGH)
    end
  endtask
endclass
```

Kareem Waseem

```
class ALSU_test extends uvm_test;
  `uvm_component_utils(ALSU_test)

  ALSU_env env;
  shift_reg_env env_shift;

  ALSU_config ALSU_cfg;
  shift_reg_config shift_reg_cfg;

  ALSU_main_sequence main_seq;
  ALSU_reset_sequence reset_seq;

  function new(string name = "ALSU_test", uvm_component parent = null);
    super.new(name, parent);
  endfunction

  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    env = ALSU_env::type_id::create("env",this);
    env_shift = shift_reg_env::type_id::create("env_shift", this);
    ALSU_cfg = ALSU_config::type_id::create("ALSU_cfg",this);
    shift_reg_cfg = shift_reg_config::type_id::create("shift_reg_cfg",this);
    main_seq = ALSU_main_sequence::type_id::create("main_seq",this);
    reset_seq = ALSU_reset_sequence::type_id::create("reset_seq",this);

    if (!uvm_config_db #(virtual ALSU_if)::get(this, "","ALSU_IF", ALSU_cfg.ALSU_vif))
      `uvm_fatal("build_phase", "Test - Unable to get the virtual interface of the ALSU from the uvm_config_db");
    if (!uvm_config_db #(virtual shift_reg_if)::get(this, "","shift_reg_IF", shift_reg_cfg.shift_reg_vif))
      `uvm_fatal("build_phase", "Test - Unable to get the virtual interface of the shift_reg from the uvm_config_db");

    shift_reg_cfg.is_active = UVM_PASSIVE;
    ALSU_cfg.is_active = UVM_ACTIVE;

    uvm_config_db #(shift_reg_config)::set(this, "env_shift.*", "CFG_SHIFT", shift_reg_cfg);
    uvm_config_db #(ALSU_config)::set(this, "env.*", "CFG_ALSU", ALSU_cfg);
  endfunction

  task run_phase(uvm_phase phase);
    super.run_phase(phase);
    phase.raise_objection(this);
    //reset sequence
    `uvm_info("run_phase", "Reset Asserted", UVM_LOW)
    reset_seq.start(env.agt.sqr);
    `uvm_info("run_phase", "Reset Deasserted", UVM_LOW)

    //main sequence
    `uvm_info("run_phase", "Stimulus Generation Started", UVM_LOW)
    main_seq.start(env.agt.sqr);
    `uvm_info("run_phase", "Stimulus Generation Ended", UVM_LOW)
    phase.drop_objection(this);
  endtask
endclass: ALSU_test
```

Kareem Waseem